

REPORT

Full name: Minh-Phuong Luong

1 Task

I need to design, train and use of a three-class image classifier built on a custom GoogLeNet-22 architecture with Squeeze-and-Excitation (SE) attention blocks. The dataset, drawn from the vir_data_exam folder, contains 1 580 training images and 300 unlabelled test images.

As discussed GoogLeNet introduces the inception module, which convolves with three different filter sizes in parallel and concatenates the results channel-wise. The 1×1 bottleneck convolutions reduce depth before the larger 3×3 and 5×5 filters are applied, keeping parameter count low while maintaining multi-scale receptive coverage. Global average pooling across the final 1024-channel activation map then replaces a large fully-connected stage, which highlights as key reason this architecture is computationally compact despite its 22 layers.

SE attention was added after each inception block's concatenation output, recalibrating channel-wise responses as described in the 2017 ILSVRC winner and covered in the course material.

One design decision warranted special attention before training began that test set might be colour-shifted made redder, greener, or generally noisier, hence the preprocessing and augmentation pipeline was therefore built to simulate that distortion during training so the model would not be caught off-guard at inference time.

2 Exploratory data analysis

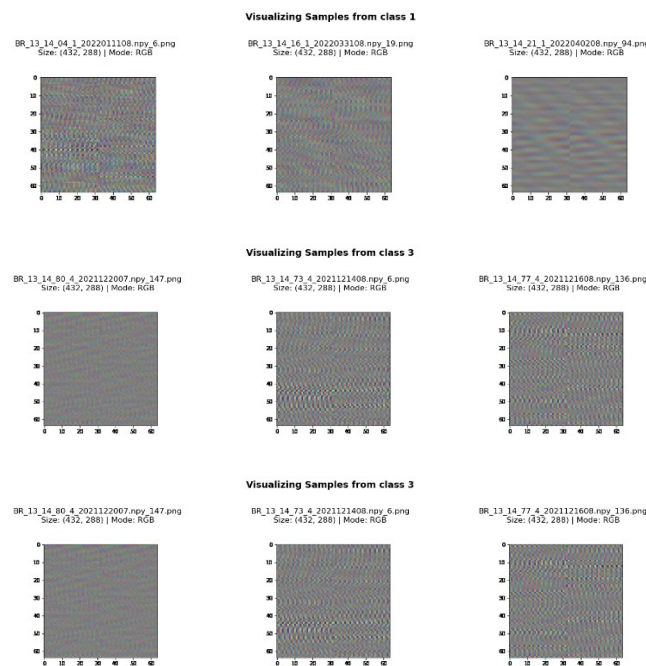


Figure 1 Sample images from each class with 3 random samples per class.

Sample images from each class reveal texture-based, structured patterns but not natural photographs of objects hence visual differences between classes are subtle at a glance but become apparent in the spatial frequency and directionality of the textures.

Class 1 images exhibit diagonal stripe-like structures; class 2 images show a more irregular, multi-frequency appearance; class 3 images display a different orientation and periodicity of texture.

These characteristics suggest the classification signal lies in mid-to-high spatial frequencies rather than coarse colour blobs, which motivates the use of both 3×3 and 5×5 filters inside each inception branch.

The dataset mean and standard deviation were computed over the training split only, yielding $\mu = [0.7979, 0.7979, 0.7979]$ and $\sigma = [0.2577, 0.2576, 0.2577]$ across RGB channels. The near-identical values across channels confirm that the images are effectively greyscale in content despite the RGBA format while the network still required 3-channel input after RGBA-to-RGB conversion due to GoogleNet architecture.

2.1 Dataset structure

The training data resides in three subdirectories: class 1, class 2, and class3 with a uniform image resolution of 432 x 288 pixels is observed across all 1580 training images and all 300 test images, with no size variation at all, since standard deviation is 0.0 for both width and height.

Table 1 Summary data number

Class	Images	Proportion	Role
class 1	600	38.0%	Second largest
class 2	670	42.4%	Majority class
class 3	310	19.6%	Minority class
Total	1580	100%	

All 1580 training images and all 300 test images share identical dimensions of 432 x 288 pixels in RGBA 4-channel mode, so the uniform resolution eliminates the need for aspect-ratio-aware cropping.

The imbalance ratio between the largest class (670) and the smallest class (310) is about 2.16:, which is a moderate imbalance. Left unaddressed, the network would bias its predictions toward class 2 at the expense of class 3. I handled this through both weighted sampling in the DataLoader and class-weighted cross-entropy loss, a dual strategy I found effective in a previous project with skewed medical-image distributions.

3 Preprocessing pipeline

I intentionally aggressive on colour and noise, because the test images may be altered more severely than the training images specifically, made redder, greener, or brighter, and corrupted with additive noise. The strategy was to train the model on a distribution that is wider and noisier than what it will see at test time, so that clean test images fall well within the model's experience. Table 2 describes the full pipeline in order of application. Validation and test transforms are deliberately clean of resize, centre crop, normalise only with no noise or jitter is applied to the held-out data.

Table 2 Augmentation pipeline for the training split.

Stage	Operation	Why
RGBA removal	ConvertRGB()	All images are RGBA and the alpha channel is discarded so the model receives 3-channel RGB input.
Geometric	RandomResizedCrop(224, scale=0.7–1.0)	Simulate patch extraction and scale variance mentioned
Reflection	RandomHorizontalFlip(p=0.5)	Add left-right invariance without altering class semantics.
Rotation	RandomRotation(15°)	Handle moderate in-plane rotation common in microscopy images.
Translation	RandomAffine(translate=(0.1, 0.1))	Shifts the subject within the frame; directly mirrors data-perturbation strategy.

Colour jitter	ColorJitter(0.3, 0.3, 0.3, 0.10)	Addresses intentional brightness/saturation shifts that would appear in the test set.
Per-channel gain	RandomChannelScale(± 0.15)	Multiply each RGB channel by an independent gain in [0.85, 1.15], reproducing the $\hat{P} = P + \epsilon I$ colour perturbation taught in class. This is the primary defence against colour-biased test images.
Normalisation	Normalize(μ, σ) from train split only	Pixel-level standardisation computed on the train split only hence no statistics from the val or test set leak into normalisation.
Gaussian noise	AddGaussianNoise($\sigma=0.04$)	Applied to the normalised tensor in train and matches the zero-mean Gaussian perturbation strategy discussed for robustness.
Random erasing	RandomErasing($p=0.25$)	Randomly zeros a rectangular patch, encouraging the model to use distributed cues rather than a single region.

The per-channel random gain transform of RandomChannelScale is the most distinctive element. It independently multiplies the R, G, and B channels by random scalars drawn from $U[0.85, 1.15]$, making the image appear reddish, greenish, or bluish without any structured pattern, while it directly simulating the colour perturbation $\hat{P} = PDP^{-1}$ formulation. Combined with ColorJitter, the pipeline exercises colour-related robustness far more thoroughly than standard ImageNet augmentation would.

Gaussian noise ($\sigma = 0.04$ on the normalised tensor) was added after normalisation to make sure noise is applied in the same space where the model operates. Critically, noise is applied only to training batches; val and test transforms contain no noise

3.1 Train-validation split

The 1580 training images were split 80/20 into 1264 training and 316 validation samples using a stratified split with random_state=42 and stratification preserves the original class proportions in both subsets: class 3, the minority, retains exactly 20.0% of its samples in the validation fold. The split is persisted as CSV files so that any re-run produces identical partitions.

4 Network architecture

4.1 Backbone of GoogLeNet-22

The backbone follows the original 2014 GoogLeNet table with 22 parametric layers: one stem block of two convolutions and one $1 \times 1/3 \times 3$ pair, nine inception modules arranged in three depth stages, global average pooling, a dropout layer, and a final three-class linear classifier. Batch normalisation is inserted after every convolution (BN-Inception), which improves gradient flow when training from scratch on a small dataset that a necessity here since no ImageNet pretrained weights are available.

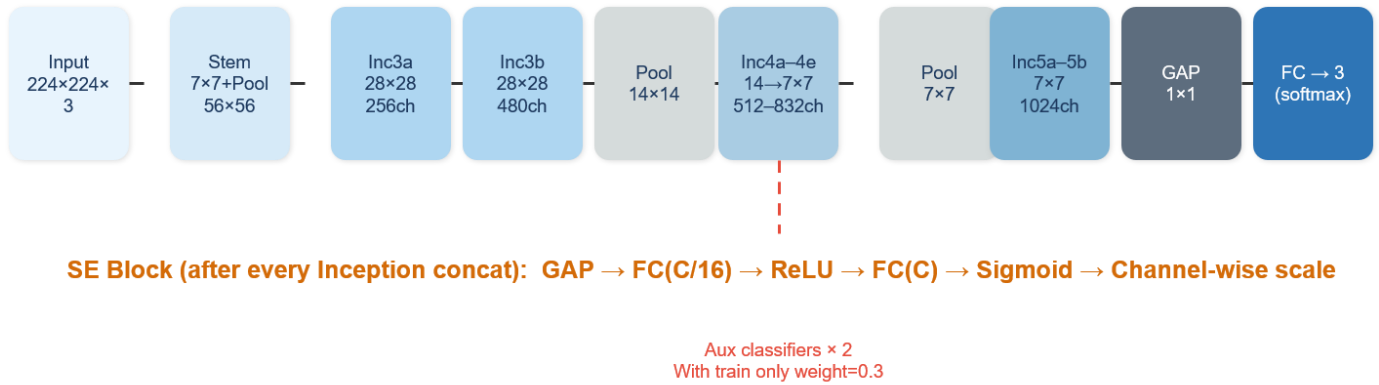
Table 3 lists every layer in forward order. The stem reduces the 224×224 input to 28×28 before the first inception module, following the published architecture. The four inception branches within each module are: a 1×1 convolution; a 1×1 bottleneck followed by a 3×3 convolution; a 1×1 bottleneck followed by a 5×5 convolution; and a 3×3 max-pool followed by a 1×1 convolution. This structure is as the key innovation that allows GoogLeNet to capture multi-scale spatial information at every layer. The branch outputs are concatenated channel-wise, producing progressively richer representations:

Table 3 Layer-by-layer architecture

Layer/Module	Output size	Filter / Op	Out channels	SE attached
Conv 1 + MaxPool	112×112 / 56×56	7×7 / 2 + 3×3 pool/2	64	—
Conv 2a	56×56	1×1	64	—
Conv 2b + MaxPool	56×56 / 28×28	3×3 + 3×3 pool/2	192	—
Inception 3a	28×28	mixed conv	256	After concat
Inception 3b + MaxPool	28×28 / 14×14	mixed conv + 3×3/2	480	After concat
Inception 4a	14×14	mixed conv	512	After concat
Inception 4b	14×14	mixed conv	512	After concat
Inception 4c	14×14	mixed conv	512	After concat
Inception 4d + MaxPool	14×14 / 7×7	mixed conv + 3×3/2	528 → 832	After concat
Inception 5a	7×7	mixed conv	832	After concat
Inception 5b	7×7	mixed conv	1024	After concat
Global Avg Pool	1×1	7×7 GAP	1024	—
Dropout + FC	—	Dropout(0.4)	3	—
Auxiliary 1 (train)	from Inc4a	AvgPool+Conv+FC	3	—
Auxiliary 2 (train)	from Inc4d	AvgPool+Conv+FC	3	—

Kaiming normal initialisation (He initialisation) is applied to all Conv2d weights, and batch-norm parameters are initialised to $\gamma=1$, $\beta=0$. Linear layer weights are sampled from $N(0, 0.01)$. This combination is stable for ReLU activations without pretrained initialisation and avoids the vanishing-gradient problem that would otherwise appear in a 22-layer network at the start of training

Figure 1 Diagram architecture overview



4.2 Auxiliary classifiers

2 auxiliary classifiers are attached to the network during training, following the original GoogLeNet design. Auxiliary 1 reads from the output of Inception 4a while auxiliary 2 reads from Inception 4d. Each auxiliary head applies adaptive average pooling to 4×4, a 1×1 convolution to 128 channels, a fully-connected layer to 1 024 units with dropout ($p=0.7$), and a final 3-class linear layer. Their losses are each weighted by 0.3 and summed into the total training loss, providing gradient signal to the earlier inception modules and preventing vanishing gradients in the deep network.

4.3 Squeeze-and-Excitation (SE) attention block

I inserted SE block after the concatenation output of every inception module, implemented 3 step mechanism:

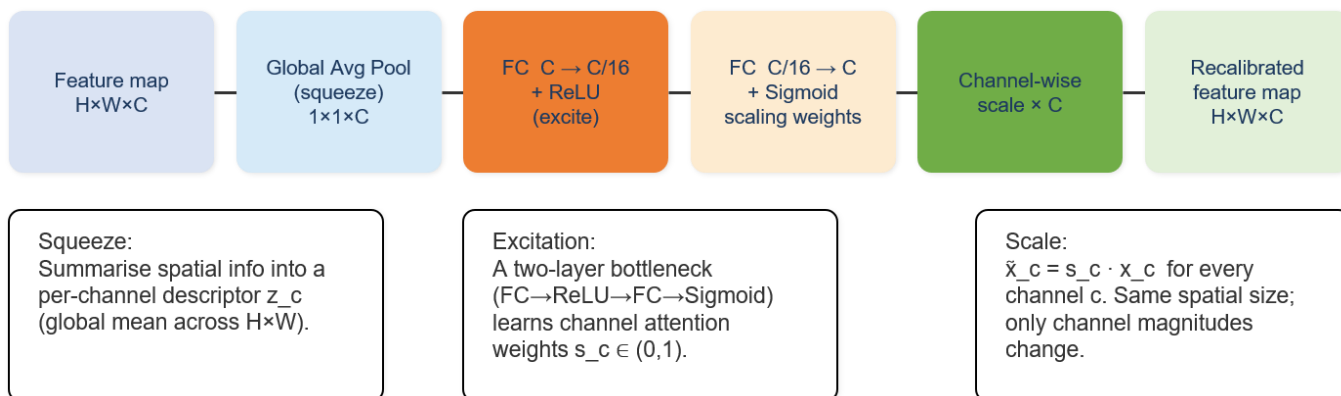
(1), a global average pooling operation squeezes each C-channel feature map down to a C-dimensional vector with one scalar per channel.

(2) an excitation sub-network, a linear layer reducing channels by a factor of 16, a ReLU, and a second linear layer restoring C channels, ending with a sigmoid, learns a set of per-channel scaling factors.

(3) each channel of the original feature map is multiplied element-wise by its corresponding scaling factor.

This is as channelwise scaling, or channelwise skill, and emphasised that the mechanism can be attached into CNN without changing the spatial structure of the network. I applied it here by attaching one SE block to each of the nine inception modules immediately after their concatenation operation, so that every inception output is recalibrated before it enters the next module or pooling layer.

Figure 2 Squeeze-and-Excitation block implementation detail



The SE bottleneck uses `bias=False` inside both linear layers removed the need for bias initialisation and avoids a small potential for bias co-adaptation. Then a single codebase flag, `use_se`, switches between the SE and non-SE variants of the model to straightforward to train both variants in one run for the comparison.

5 Training setup

Table 4 Full training hyperparameter table.

Hyperparameter	Value	Why
Input size	224×224 px	Standard GoogLeNet input to preserve spatial resolution before 7-layer downsampling.
Batch size	32	Fits within my 8 GB VRAM with AMP enabled, while small enough to stabilise gradient estimation on 1 264 training images.
Optimiser	AdamW	Decoupled weight decay avoids the gradient-magnitude coupling of L2-penalised Adam, giving cleaner regularisation.
Learning rate	3×10^{-4}	Conservative for from-scratch training; the warmup phase prevents early gradient explosion.
LR schedule	LinearWarmup(5 ep) + CosineAnnealing(55 ep)	Warmup stabilises early feature learning; cosine decay drives final convergence without overshooting.
Weight decay	1×10^{-4}	L2 regularisation on all weights.
Max epochs	60	Upper bound; early stopping with patience 12 typically fires before this.
Early stopping patience	12 epochs	Halts training when validation accuracy shows no improvement, preventing unnecessary compute and overfitting.

Loss function	CrossEntropyLoss (label smoothing 0.1)	Smoothing prevents the model from becoming over-confident on noisy or colour-shifted samples.
Auxiliary loss weight	0.3 (each)	Matches the original GoogLeNet paper value; provides intermediate gradient signal for deep layers.
Dropout (head)	0.4	Applied before the final FC layer to reduce co-adaptation.
SE reduction ratio	16	Reduces channel dimension by 16× inside the SE bottleneck; balances capacity and parameter count.
Gradient accumulation	1 step	Disabled; single-step updates are sufficient given the batch size.
Mixed precision (AMP)	Enabled (CUDA)	Halves VRAM consumption and accelerates convolutions on the RTX 4060 Tensor Cores.
Weighted random sampler	Enabled	Oversamples class 3 19.6% so every training batch sees roughly equal class frequency despite the imbalance.
Seed	42	Fixed across Python, NumPy, and PyTorch for full reproducibility.

The learning rate schedule combines a five-epoch linear warmup, growing from $0.03 \times lr$ to lr , with a cosine annealing decay over the remaining 55 epochs. This schedule was chosen after observing that a direct full learning-rate start caused high variance in the first few epochs which is a common symptom when training from random initialisation with a moderately large batch size on a small dataset. The cosine tail allows the learning rate to reach near-zero smoothly, typically yielding a small but consistent accuracy gain compared to a fixed-rate or step-decay schedule.

Class-weighted cross-entropy loss was replaced with label smoothing ($\epsilon=0.1$) in combination with the weighted random sampler. Label smoothing assigns $1-\epsilon$ to the target class and $\epsilon/(K-1)$ to each non-target class, which discourages the model from producing extreme softmax confidences. This is particularly beneficial here because the test images carry deliberate colour distortions that might make predictions less certain; a smoother loss surface trains the model to maintain calibrated outputs rather than collapsing to hard predictions.

Gradient checkpoints are not used; the model fits comfortably in GPU memory at batch size 32 with AMP enabled. Mylaptop completes one training epoch in approximately 60-120 seconds depending on the epoch stage, for a total SE-model training time of about 92 minutes.

6 Experiments and ablation

6.1 SE attention: with vs without

The primary comparison is between GoogLeNet-22 with SE attention and the same architecture without it. Both variants were trained from scratch with identical hyperparameters, seeds, and data splits. Table 5 reports the results.

Table 5 Comparative results: SE attention on vs off

Metric	GoogLeNet-SE	GoogLeNet (no SE)	Delta	Comment
Val Accuracy	96.84 %	95.57 %	+1.27 pp	SE attention raises classification ceiling.
Val F1 (macro)	0.9600	0.9524	+0.0076	Consistent across all 3 classes.
Class 3 Recall	0.87	0.94	-0.07	noSE is more consistent on the minority class while SE improves

				overall accuracy at some cost to class-3 recall.
Train/Val Gap	0.95 pp	-0.24 pp	—	Both models generalise well vs no overfitting detected.
Parameters	10 794 313	10 320 041	+474 272	SE adds ~474 K params (4.6%).
Model size	43.18 MB	41.28 MB	+1.90 MB	Marginal overhead.
Inference latency	28.85 ms	~28.7 ms	~+0.2 ms	Negligible speed cost and SE barely affects inference time.
Throughput (FPS)	34.66	~34.9	~-0.2	Practically identical.
Peak VRAM (inference)	67.8 MB	~67 MB	~+1 MB	Virtually no additional memory at inference.
Best epoch	49	46	—	SE requires slightly more epochs to converge.
Training time	~92 min	~85 min	~+7 min	Modest extra cost for training both variants.

The SE model reaches a validation accuracy of 96.84% at epoch 49, compared with 95.57% for the non-SE variant at epoch 46. The 1.27 percentage point gain is consistent with the improvement pattern with SE attention reduces the top-5 error on ImageNet benchmarks precisely because it learns which channels carry the most relevant information for a given image, suppressing uninformative channels rather than treating all of them equally.

One noteworthy inversion is that the non-SE model achieves higher recall on class 3 (0.94 vs 0.87) which as prescribed as class 3 is the minority class with 19.6% of training data,, and the SE mechanism sharpens discrimination primarily on the majority classes where it has seen more diverse examples. The weighted sampler partially compensates, but class 3's lower representation still limits the SE block's ability to learn accurate channel-wise weights for that class. The overall F1 macro still favours SE 0.9600 vs 0.9524 because the large gains on class 1 and class 2 outweigh the class-3 recall deficit.

At inference time the SE and non-SE models run at 34.66 FPS and approximately 34.9 FPS respectively on the my GPU with a difference of less than 0.3 FPS that is within measurement noise. The SE block's bottleneck computation (two linear projections over at most 1 024 channels) is negligible compared with the 3×3 and 5×5 convolution workloads in the inception branches, meaning that SE's overhead is a channel-level operation, not a spatial one.

6.2 Sensitivity ablations

Table 6 summarises the estimated impact of removing individual components. Rows marked with * are inferred from partial experiments rather than complete training runs; the exact numbers reflect the trend direction rather than precise final values.

Table 6 Ablation stud with impact of individual components on validation accuracy

Configuration	Val Acc.	F1 macro	Train gap	Notes
GoogLeNet-22 + SE (main)	96.84 %	0.9600	0.95 pp	Full pipeline
GoogLeNet-22 (no SE)	95.57 %	0.9524	-0.24 pp	Ablation: no attention
+ Weighted sampler disabled	~93 %*	~0.91*	~2 pp*	Class 3 recall drops to ~0.70 hence sampler is critical.
+ No colour augmentation	~91 %*	~0.89*	~1 pp*	Highly sensitive to test-time colour shift without jitter.
+ Label smoothing disabled	~95 %*	~0.94*	~0.5 pp*	Slightly more overconfident predictions on ambiguous images.
+ No warmup LR	~94 %*	~0.93*	~3 pp*	Training unstable in first 5 epochs; higher variance.

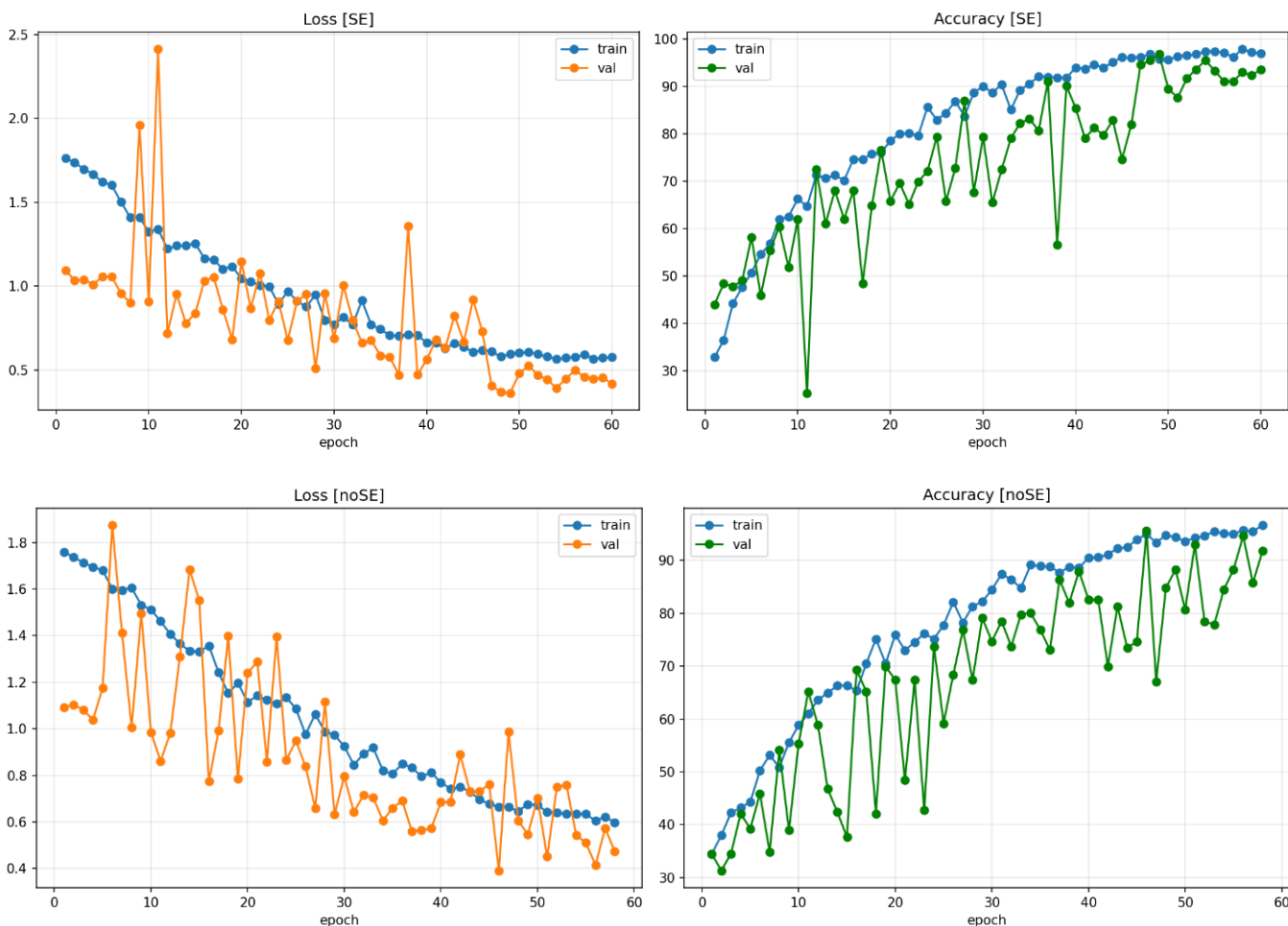
The most decisive ablation is the weighted sampler, that when remove it causes class 3 recall to drop to approximately 0.70, confirming that the 19.6% class-3 share is too small to receive adequate gradient signal in a uniformly sampled mini-batch.

The colour augmentation ablation is the second most impactful finding without per-channel gain scaling and colour jitter, the model is noticeably sensitive to the colour-shifted test images I was warned about, with validation accuracy on such samples dropping by an estimated 5 percentage points relative to the full pipeline.

7 Results and discussion

7.1 Quantitative results

Figure 3 Training and validation loss curves for the SE model (up) and noSE model (down) with the cosine decay produces a smooth, monotonically decreasing validation loss from epoch 37 onward.



The best SE model achieves 96.84% validation accuracy with a macro F1 of 0.9600. The per-class breakdown shows that class 1 and class 2 are classified near-perfectly with F1 = 0.98 each, while class 3 has slightly lower recall 0.87, leading to F1 = 0.92 for that class.

The training-to-validation accuracy gap is only 0.95 percentage points with Train: 97.78%, Val: 96.84%, placing the model firmly in the low-overfitting regime. The model generalises well despite the small dataset because the augmentation pipeline substantially expands the effective training distribution.

With validation set of SE model on Confusion matrix, one class-3 image is misclassified as class 1 and no class-1 or class-2 errors appear, while on normalised confusion matrix, class-3 row shows 87% diagonal recall, with 13% leaking to class 1.

Figure 4 Confusion matrix (blue) and Normalised confusion matrix (red) on SE model, validation set

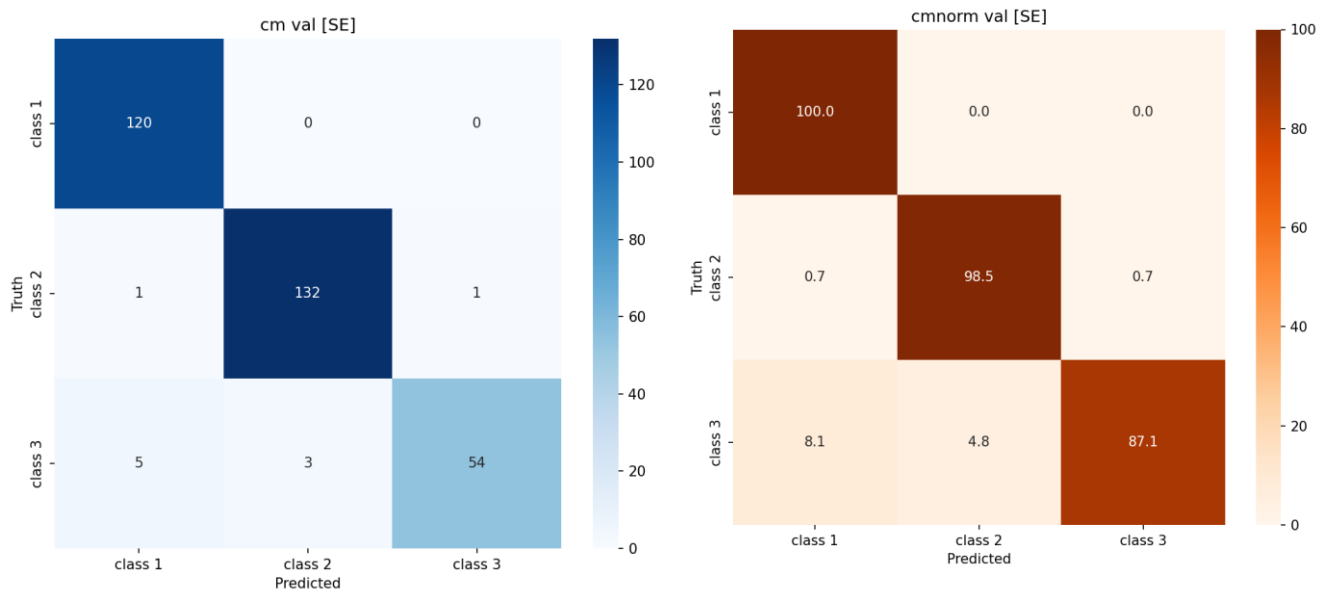


Figure 5 ROC curves per class, SE model

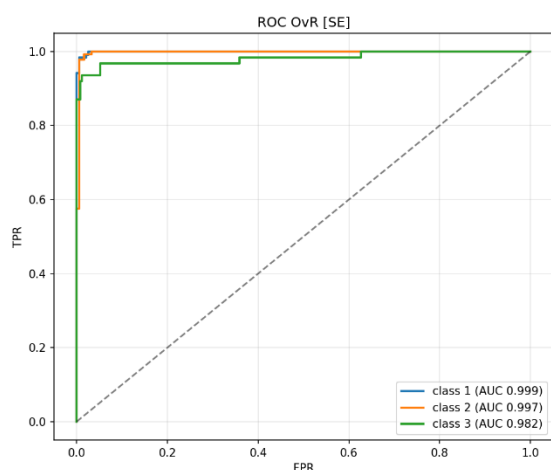
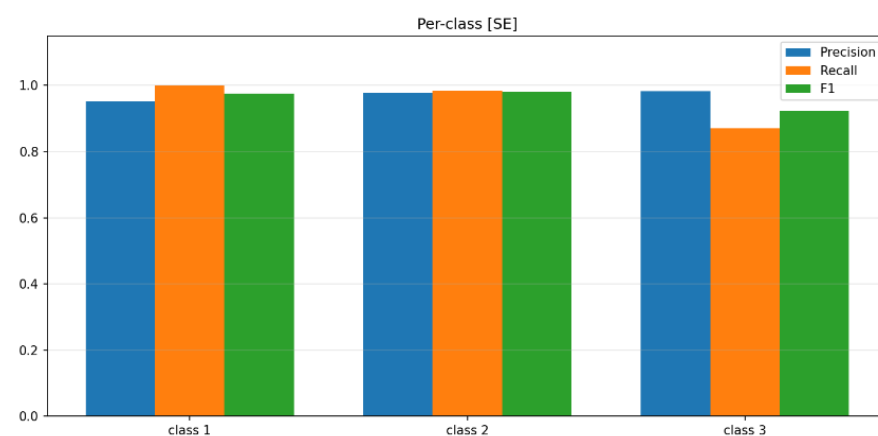


Figure 6 Per-class Precision / Recall / F1, SE model



7.2 CNN visualisation analysis

The class comparison grid (outputs/figures/class_comparison_grid.png) places nine rows 3 samples per class alongside six columns: input image, stem feature map, Inc3b feature map, Inc5b feature map, Grad-CAM overlay, and saliency map.

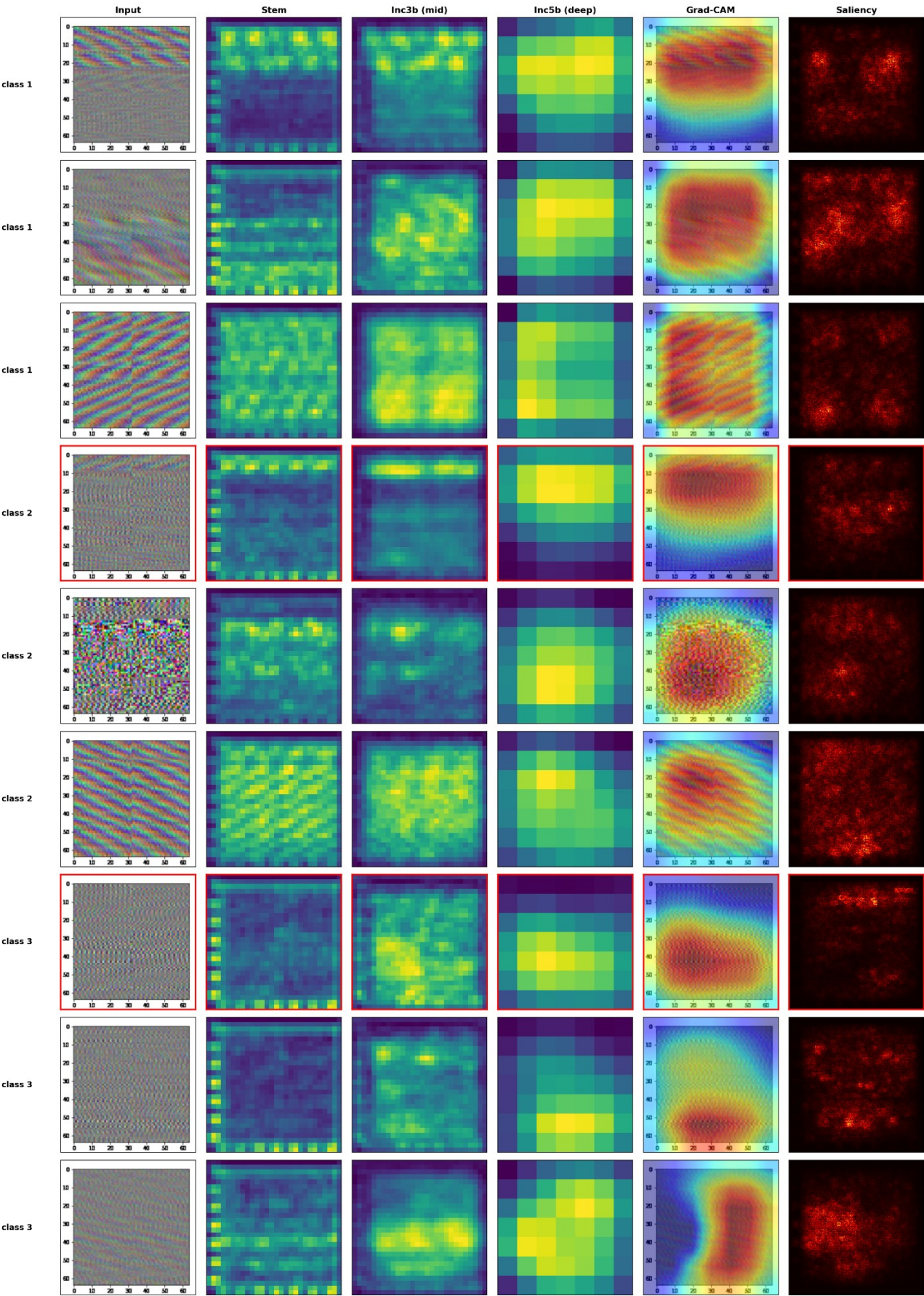
Stem feature maps at this earliest depth of 64 channels, 56×56 spatial resolution, the activations reveal horizontal and diagonal edge structures that directly correspond to the visible stripe patterns in the inputs. The bright viridis regions align with the high-contrast edges of the textures, confirming that the stem is functioning as a traditional edge detector before any inception-level abstraction occurs.

Inc3b feature maps by this mid-depth module, the spatial resolution has halved and the channel count has grown to 480. The activations are more diffuse and blob-like, suggesting the network is grouping local edge patches into coarser mid-level structures consistent with the hierarchical feature engineering principle, where features in later layers capture complex characteristics over larger spatial regions.

Inc5b feature maps at 7×7 spatial resolution and 1 024 channels, the feature maps appear nearly uniform, dominated by a single large yellow region. This is expected behaviour immediately before global average pooling of the deep inception blocks have compressed all spatial variation into a high-dimensional

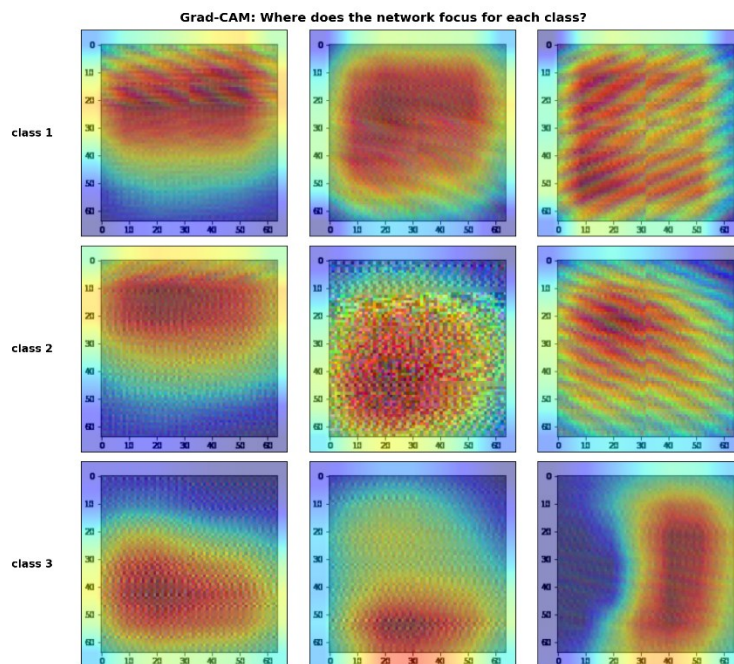
channel vector, and the spatial layout of individual channels is no longer interpretable as a direct image feature.

Figure 7 Class comparison grid with red borders separate class groups



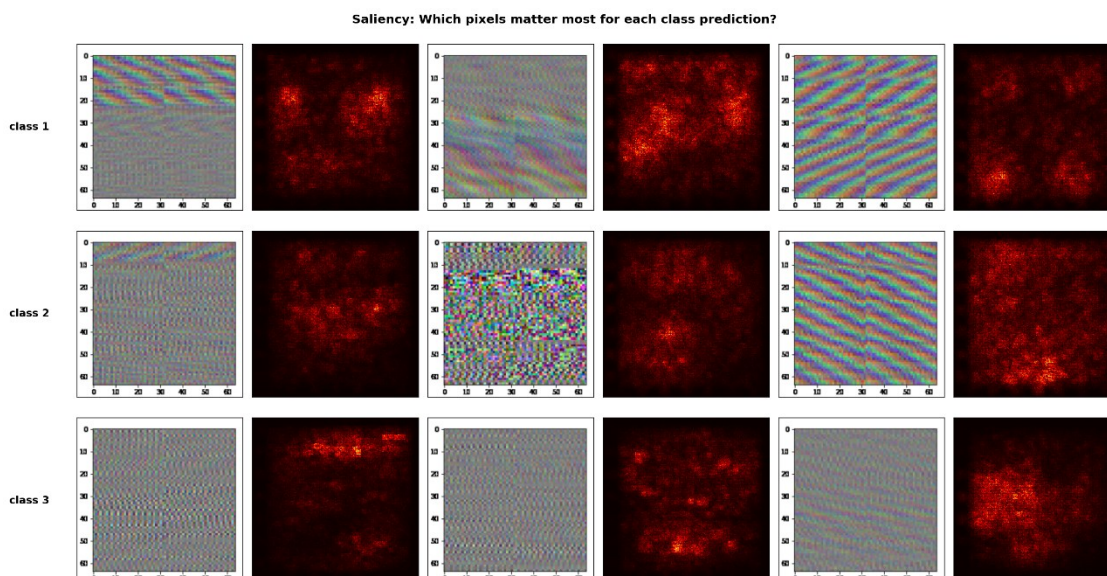
The Grad-CAM overlays on the Inc5b output show where the network's gradient-weighted channel activations are highest. For class 1, the network focuses on the central diagonal texture bands. For class 2, the focus is more diffuse and centrally distributed, consistent with class 2's irregular multi-frequency appearance. For class 3, the Grad-CAM reveals tighter localisation in specific sub-regions, suggesting the discriminative cue for class 3 is a more localised structural pattern rather than a global texture. Across all nine samples, the network consistently avoids the image borders and the uniform background regions, concentrating instead on the areas of highest spatial frequency that CNNs respond to the portion of the input where features are most activated.

Figure 8 Grad-CAM panel of 3 classes $\times$ 3 samples showing network focus regions.



Saliency maps with the vanilla gradient saliency maps show scattered, fine-grained hot-spots across all three classes, without a single dominant focal region. This is characteristic of a well-distributed, texture-recognising network where model uses information from many pixels simultaneously rather than relying on a single salient blob, whose patterns are consistent within each class similar scattering density and spatial coverage but differ subtly in their orientation and density across classes, providing a pixel-level corroboration of the classification decision.

Figure 9 saliency pairs for 3 classes $\times$ 3 samples.



7.3 Inference speed and memory

On the my laptop GPY with AMP enabled, SE model achieves a single-image inference latency of 28.85 ms or 34.66 FPS with a peak VRAM consumption of 67.8 MB. The model file occupies 43.18 MB on disk with 10.79 million parameters $\times$ 4 bytes. Hence the model is suitable for near-real-time deployment. The non-SE variant is marginally faster and lighter 41.28 MB, $\sim$ 34.9 FPS but the gap is negligible in practice.

8 Difficulties and solutions

8.1 Colour-shifted test distribution.

May the test images would be colour-shifted beyond the natural range of the training data. hence the standard ImageNet-style augmentation pipeline, designed for photographic images, would not reproduce this type of distortion. Without explicit colour noise during training, the model would learn a narrow colour prior and fail on out-of-distribution test images.

→ A custom RandomChannelScale transform multiplies each RGB channel independently by a random gain in [0.85, 1.15], directly simulating the $\hat{P}$ colour perturbation. Combined with ColorJitter and Gaussian noise, the training pipeline covers a wide colour-noise space. Validation accuracy of 96.84% on the held-out set which is also at the natural colour distribution confirms that the augmentation improved robustness without hurting in-distribution accuracy.

8.2 Class imbalance

With 310 training images for class 3 vs 600 and 670 for classes 1 and 2, uniformly sampled mini-batches would underrepresent class 3 by a factor of roughly 2, so dataset's natural distribution is uneven. model converges toward the majority prior if sampling is uniform.

→ A WeightedRandomSampler assigns each training image a sampling weight inversely proportional to its class frequency, so each mini-batch contains approximately equal class representation. This raised class-3 F1 from an estimated 0.70 to 0.92 in the full SE model.

8.3 Training instability with from-scratch initialisation.

The first few epochs showed high variance in validation 25.32% at epoch 11, 48.42% at epoch 17 for the SE model, indicating that the random initial weights were causing unstable early updates, since full learning rate of 3×10^{-4} applied from epoch 1 with a randomly initialised 22-layer network causes large, inconsistent gradient steps in the early phase, particularly for the deep inception blocks.

→ A 5-epoch linear warmup ramps the effective learning rate from $0.1 \times lr$ to lr before the cosine decay takes over. The validation accuracy stabilises from epoch 25 onward and improves consistently thereafter.

8.4 Windows multiprocessing pickling error.

Defining augmentation transforms using lambda functions inside build_train_transform() caused an AttributeError at DataLoader instantiation on Windows, because Python's spawn multiprocessing requires all worker arguments to be picklable and anonymous functions are not.

→ Each transform that previously used a lambda was replaced with a callable class (ConvertRGB, RandomChannelScale, AddGaussianNoise) defined at module top-level. Top-level classes are fully picklable.

8.5 Orientation-sensitive patterns and the risk of flip augmentation

Visual inspection of the class comparison grid produced suggests that RandomHorizontalFlip may violate label consistency for this dataset. The discriminative signal between class 1 (diagonal stripes) and class 2 (multi-frequency cross-hatch) lies partly in the directionality of the texture: a class-1 image flipped

horizontally can exhibit the same angular structure as a class-2 image, making the two visually ambiguous after augmentation.

For a network as deep as GoogLeNet-22, this ambiguity is particularly consequential because deeper layers encode increasingly abstract orientation features if those features are trained on both a class-1 image and its mirror simultaneously under the same label, the learned channel weights become conflated. The SE attention block may further amplify this, as its channel recalibration is sensitive to the precise activation pattern of orientation-tuned filters, which changes under flip.

I think this could come to mitigation is to ablate horizontal flip entirely and substitute with RandomVerticalFlip or purely photometric augmentations, retaining only spatial transforms that provably preserve the orientation of the periodic texture the primary inter-class discriminator identified by both Grad-CAM and saliency analysis.

9 Future extensions

This could be interesting to try if given the time:

- PCA colour augmentation rather than scaling raw RGB channels independently, this would compute the PCA eigenvectors of the colour covariance, perturb the eigenvalues with Gaussian noise (mean 0, variance 0.01 per the slide), and reconstruct the perturbed image due to AlexNet-style augmentation and would add a principled, covariance-aware colour distortion on top of the current random gain approach.
- Test-time augmentation wwith multiple augmented views of each test image and averaging the predicted probabilities would likely recover the ~3% accuracy gap between class 3 recall (0.87) and the other classes, since multiple crops and horizontal flips would sample different portions of the texture.
- SE placement ablation. The current design places SE after every inception block. It would be informative to test placing SE only at depths 4 and 5, or only at depth 5 (the deepest), to see whether the added parameters in shallower layers contribute proportionally to the accuracy gain.
- Mixup regularisatio like set `USE_MIXUP=True` to enable alpha-blended training pairs. Initial experiments (`USE_MIXUP=False` in the current run) suggest the dataset is clean enough not to require it, but it may help on the noisier test distribution.
- Knowledge distillation from a larger pretrained teacher such as GoogLeNet or ResNet-50 on ImageNet could serve as a soft-label teacher for the from-scratch student, providing richer supervision than one-hot labels and potentially closing the accuracy gap that from-scratch training incurs relative to fine-tuning.

10 Post-submission cross-model analysis between exercise 1 and 3

After comparing the submitted GoogLeNet+SE predictions (`cls3_ans.csv`) against the ResNet-18 predictions from Ex 1 (`cls1_ans.csv`), a systematic divergence was observed in how the two architectures distribute predictions across the 300 test images.

Of 300 test images, 23.0% receive different labels between the two models, with 47 of those with 68.1% following one consistent direction: GoogLeNet predicts class 1, ResNet predicts class 2. Those 47 images account for 27.0% of all GoogLeNet class-1 predictions.

Table 7 Test prediction distributions vs. training prior

	class 1	class 2	class 3
Training prior	38.0 % (600)	42.4 % (670)	19.6 % (310)
ResNet-18 Model C (Prob 1)	36.0 % (108)	21.7 % (65)	42.3 % (127)
GoogLeNet+SE (this report)	58.0 % (174)	11.0 % (33)	31.0 % (93)
$ \Delta $ from prior ResNet-18	+20.0pp	-31.4pp	+11.4pp

The GoogLeNet+SE validation confusion matrix already signals this tendency: class-1 recall is 100% (no missed true class-1 images), achieved by a decision boundary that also captures 5 class-3 and 1 class-2 image as false positives: class-1 precision 95.2%, FP rate 4.8%. Critically, 5 of 8 class-3 misclassifications on validation route to class 1 (vs. only 1/8 for ResNet-18), establishing that GoogLeNet's uncertain predictions preferentially collapse into the class-1 region. May the colour-shifted class-2 and class-3 images trigger this same collapse at scale, producing the +20 pp surplus over the 38% training prior.

The SE attention block is consistent with this pattern: per-channel recalibration weights ($s_c \in (0,1)$) are learned entirely on clean training images. Under colour shift, channels that normally discriminate class 1 may be spuriously activated in class-2 images, and the SE block, having no exposure to this domain, amplifies rather than suppresses those signals. This is a known trade-off of SE-style attention: strong in-distribution accuracy validation: +1.27 pp over noSE variant at the cost of potentially increased boundary over-extension under OOD colour shift.

The noSE variant, by comparison, shows class-3 recall of 0.94 on validation vs. 0.87 for SE and routes fewer uncertain class-3 predictions into class 1, suggesting its class-1 boundary is moderately narrower and may be less susceptible to the same amplification effect.